開發者 CAFÉ • 2026-09-29

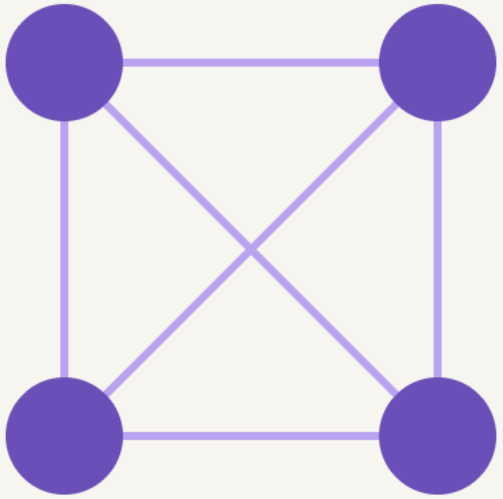
我幫筆記養了一隻精靈



一個 Obsidian 插件，
和它逼我想清楚的一個問題

Rex｜軟體工程師
平常在做 AI、開發者工具相關的東西。

我以為我的筆記是一張網



我以為的

2463 篇 • 2559 條連結
抽 300 篇，每篇拿語意上最近的十篇，
去問連結圖：四跳以內走得到嗎？

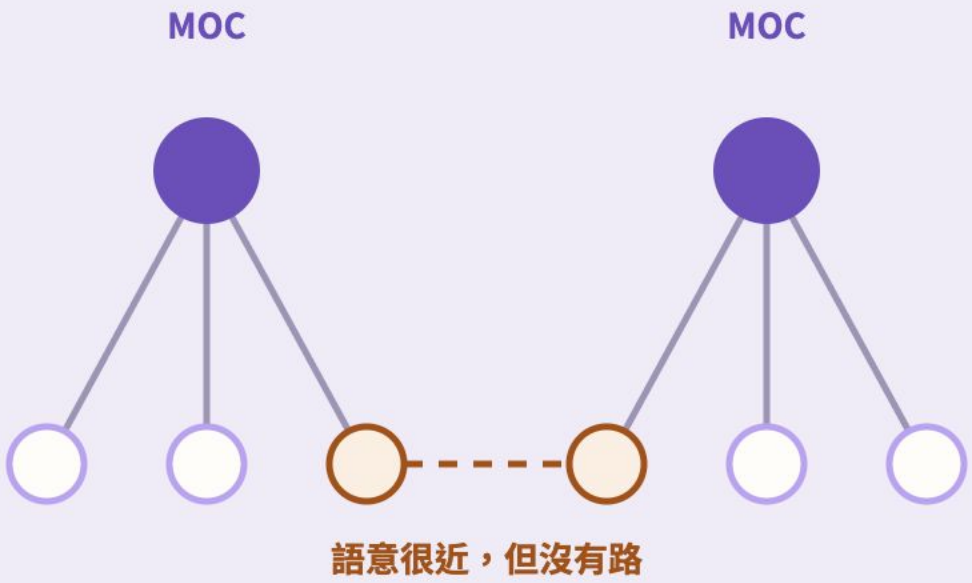
實際量到的（9/21）	比例
已經連著	4.4%
隔 2 跳	28.8%
隔 3～4 跳	8.6%
走不到	58.2%

它不像我想像中的網，比較像一棵一棵局部長出來的樹。

MOC 讓我找得到，但沒讓我碰得到。

MOC ＝ Map of Content：目錄型筆記，只放連結

同一棵樹：經過 MOC，隔 2 跳走得到。
不同棵樹：再像也走不到。



01 • 為什麼做

所以我缺的，是兩種東西

我以前留下過的東西——

🧠

寫過，但現在想不起來

✨

兩篇我都知道，
但從沒把它們連起來

← 我真正想要的

我想要的其實很簡單：**隨便記，不怕找不回來，不用花太多時間整理。**

02 • 我怎麼記筆記

建一篇筆記，要先回答它是什麼

1
點左邊的按鈕
Commander

2
先選 subtype
Idea、Tool、Terminology…
QuickAdd

3
取名字

4
自動歸位
加前綴、套範本、搬到指定資料夾
Templater

在內文打 [[新筆記]] 建連結，點下去也走同一套。

03 • 除了連筆記，它還能做什麼？

三種模式

🧠

我寫過這句

不用整理，也找得回來

✨

亂接一下

兩篇碰在一起，長出新想法

👀

我注意到

懂我在幹嘛，陪著我

模式一 · 🧠 找回

我打一句話，它說我寫過

一個字一個字打

我認為義大利麵就應該拌42號混凝土▌

比的是你剛打的**最後 15 個字**，
逐字對整個筆記庫。

這一種**完全不用 LLM**。

模式一 · 🧠 找回

還有另一種：意思像也算



意思像

這段換成向量，找最像的段落，
再請模型判：
是同一個想法嗎？是你寫的嗎？

模式二 • ✨ 新想法

兩篇我從沒連過的筆記

2026-03

Karpathy 講
「直覺程式開發」Vibe Coding

2026-04

Stanford 講座：
AI 時代的軟體開發與職涯

✨ 它說兩篇卡在同一件事

程式寫得越快，難的就越不是寫程式，
是決定要寫什麼。

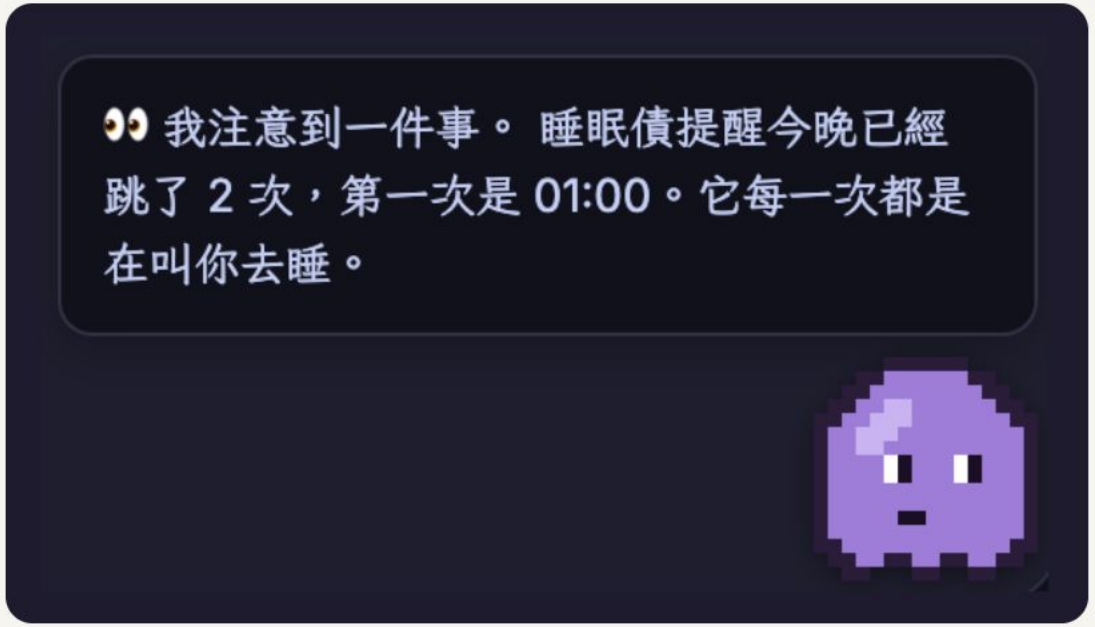
模式二 • ✨ 新想法 • 點開泡泡

它給的不是答案，是一個問題

都卡在	AI 讓寫程式變快時，工程師的重心會從產碼轉向判斷要做什麼產品
這邊寫	「工程師角色正從純粹的程式碼撰寫者轉變為產品工程師，更注重產品思維、使用者需求及產品迭代。」
另一邊寫	「當程式碼產出速度極快，最大的瓶頸不再是『工程能力』，而是『產品管理（PM）能力』。」
所以	**當做出產品變得更容易，怎麼判斷哪個使用者需求值得投入？**
名字	瓶頸轉移

模式三 • 👀 陪伴

它一路看著我





這些數字全是**數出來的**，不是模型猜的。

準確的關聯，我自己就知道了。
我要的是讓我「咦」一下。

代價是：它常常什麼都講不出來。

因為我允許它回答「這兩篇其實沒關係」——
而它大部分時候，真的這樣回答。

04 • 它怎麼做的

一個插件，沒有想像中神祕

三個檔

`manifest.json`
`main.js`
`styles.css`

一個 class

`onload()`
`onunload()`

跟 Obsidian 借東西

命令、側邊欄、
檔案讀寫、筆記索引

① 開資料夾
筆記庫的 .obsidian/plugins/ 底下

② 放三個檔
就是上面那三個

③ 重載，在設定裡打開
Obsidian 讀 main.js、執行它、呼叫 onload

04 • 它怎麼做的

最小的插件：打開就跳一句 Hello world

manifest.json

```
{
  "id": "hello-world",
  "name": "Hello World",
  "version": "1.0.0",
  "minAppVersion": "1.0.0"
}
```

styles.css

這個例子用不到，可以不放

main.js

```
const { Plugin, Notice } = require("obsidian");

module.exports = class Hello extends Plugin {
  onload() {
    new Notice("Hello world");
  }
};
```

跟 Obsidian 要工具 → 交出一個 class → 打開時跳通知

放進 .obsidian/plugins/hello-world/ → 重載 → 打開：右上角跳出 Hello world。

不用 TypeScript、不用編譯，main.js 可以直接手寫。

Obsidian 其實是一個瀏覽器視窗

Chromium

畫畫面、跑 JavaScript
（跟 Chrome 同一個核心）

＋

Node.js

讀寫電腦上的檔案、
開別的程式

＝

Electron

Obsidian 就是用它做的
VS Code、Slack 也是

所以一個插件，就是一段跑在這個視窗裡的 JavaScript。

04 • 它怎麼做的

打開 Obsidian 時，它對每個插件做三件事

① **讀 manifest.json**
知道這個插件叫什麼、第幾版

② **把 main.js 當成一段文字讀進來，執行它**
執行時，你的程式可以跟它要東西：「給我 obsidian」→ 拿到它的工具

③ **拿到你交出來的 class → new 一個 → 呼叫 onload**
從這一刻起，你的程式就在 Obsidian 裡面跑了

04 • 它怎麼做的

「呼叫 onload」：你寫內容，Obsidian 決定什麼時候跑

網頁

```
button.onclick = () ⇒ { … }
```

你寫好內容，
按鈕被按下去的時候，瀏覽器替你執行

Obsidian 插件

```
async onload() { … }
```

你寫好內容，
插件被打開的時候，Obsidian 替你執行一次

你不自己執行它。Obsidian 替你執行，這就叫「呼叫」。

04 • 它怎麼做的

跟 Obsidian 要東西，有兩個時間點

② 執行你的 main.js

```
import { Plugin }
  from "obsidian"
```

拿到「零件」

插件的父類別、跳通知的工具…
這時還沒有任何一個 Obsidian 可以操作

→

③ new 你的 class

```
new 你的插件(
  app, manifest)
```

把「機器」交給你

正在跑的這一個 Obsidian，
之後叫做 this.app

→

③ 呼叫 onload

```
this.addCommand(…)
this.app.vault.read(…)
```

操作「這一台」

往這個 Obsidian 加指令、
讀這個筆記庫的檔案

② 拿到零件，③ 拿到機器。

Obsidian 叫一次 onload，你在裡面跟它要東西

```
export default class 我的插件 extends Plugin {
  async onload() {
    this.addCommand(…)       // 在命令面板加一個指令
    this.registerView(…)     // 在側邊加一個面板
    this.app.vault           // 讀寫筆記檔案
    this.app.metadataCache   // 查每篇筆記的標籤、連結
  }
  onunload() { }
}
```

Plugin 是 Obsidian 給的父類別：繼承它，就拿到這些「跟 Obsidian 要東西」的方法。
想做的話：官方的範例插件 clone 下來就能開始。

04 • 它怎麼做的

插件和 Obsidian 之間，沒有一道牆

你的程式跑起來之後，跟 Obsidian 自己的程式在同一個地方。
所以 Obsidian 能做的，插件都能做：

讀你所有的檔案

改 Obsidian 的畫面

在背景開別的程式
← 精靈就是這樣問 GPT 的

反過來說：你裝的每一個社群插件，也都有這個權力。
手機版沒有 Node.js，開不了別的程式，所以精靈只支援桌面版。

04 • 它怎麼做的

精靈怎麼問 GPT：不打 API，開一個程式

1
兩篇筆記＋提示

→

2
在背景開 Codex CLI
唯讀、空資料夾、
不給它動手的工具

→

3
從標準輸入餵進去

→

4
一行一行 JSON 讀回來
拆成那張表

好處

登入、權杖、額度都是 Codex 自己的事。
插件從頭到尾碰不到金鑰。

代價

Codex 每次都自帶約一萬 token 的自我介紹。
想一張表大約 6–11 秒。

05 · 有用嗎

剛剛的 demo 只證明它做得到。不證明它有用。

05 • 有用嗎

我拿自己做了一個盲測

30 對筆記

15 對：演算法（向量相似度）挑的
15 對：亂挑的
洗牌，我不知道哪對是哪組

每題只答一件事

有沒有產生一個
「看之前沒有、寫得下來」的想法？

有的話——現場寫出那句話。

05 · 有用嗎

第一次覺得有用，然後我自己砍掉一半

7/15 → **4/15**

第一次看（演算法挑的）

逼自己寫出新想到什麼

~~兩篇本來就在同一個專案底下~~

→ **早就知道**

~~兩篇都是同一個工作專案，我早就收斂了~~

→ **早就知道**

~~想起以前網路上認識的一個人~~

→ **回憶，不是想法**

05 · 有用嗎

換了講法，同一批再看兩輪

演算法挑的

11/15

亂挑的

4/15

三輪累積，有產生新想法的。這不是 11 次獨立實驗——同一批我看了三次。

① 判的人是我自己，
不是雙盲

② 亂挑的也不是零：
15 對裡有 4 對讓我想到東西

以前它會講幹話：我沒給它說「沒有」的權利

「找出這兩篇的關聯」

↑ 這句話裡沒有「找不到」這個選項 → 模型非找不可

A	「承擔 Bug 責任（背鍋）」	一個工作風險
B	「Andrew Ng 與 Laurence Moroney 共同主講」	一份講者名單
橋	**「各自提供了不同的權威來源」**	**空話**

05 • 有用嗎

現在：先判斷，最後才給名字

1
判斷
成立／勉強／
不成立

2
都卡在
兩篇共同
卡住的事

3
兩邊原文
這邊寫…
另一邊寫…

4
所以
一個新問題

5
這個關聯叫什麼
最後才給

而第一步，可以回答「沒有」。

備用 · 三輪的帳

同一批 30 對，只累加「這輪才有」的

哪一輪	累加
第一輪：只把兩篇擺在我面前（演算法 7、亂挑 1）	8
第二輪：加上精靈那一句＋一段長解釋，看了才想到	+3
光看精靈那一句就夠的	0
第三輪：先問「是不是同一類」，剩下的 16 對裡想到	+4
精靈自己說「看不出關係」	4 對
累積（演算法 11、亂挑 4）	15 / 30

演算法比亂挑多 7 對；換講法多救回 7 對。
（兩個 7 量的不是同一件事，只是剛好一樣多）

06 • 最難的是閉嘴

它在找哪一種關聯

	結構上已經連著	結構上沒連
語意很像	無聊，早就知道	★ 你漏掉的連結 剛剛 ✨ 那對在這格
語意中等	普通	★★ 跨界碰撞
語意不像	怪	☆ 純瞎連

右邊那一整欄，才是我要的。

06 • 最難的是閉嘴

誰管講什麼，誰管什麼時候講

你在 Obsidian 做的事：開哪篇、打幾個字、切去別的 app

↓

從 log 算出事實：回來第 4 次、待了 102 分鐘……不是模型猜的

↓

候選清單：三種模式各自丟一句想講的話進來

🧠 **模式一 找回**
從：你正在打的字，
去對全部舊筆記

✨ **模式二 新想法**
從：你停著的這篇，
它挑另一篇來配對

👀 **模式三 陪伴**
從：上面從 log
算出的事實

⇒ 講什麼

↓

閘門：每 5 秒問一次「現在能開口嗎？」預設答案是不能

⇒ 什麼時候講

06 · 最難的是閉嘴

閘門：閉嘴是預設，全部都過才講

⛔ 精靈在睡覺
⛔ 你正開著命令面板
⛔ 今天講滿 8 次了
⛔ 25 分鐘冷卻還沒過
⛔ 你還在打字（要停 8 秒）
⛔ 你 10 分鐘沒動，人應該不在
⛔ 上次開口後只寫了 40 字，不到 120
⛔ 這篇太短

✅ 全部通過，才講

泡泡 3 秒內被關掉 → 冷卻 ×2
看完才關 → 冷卻 ÷2

它不會變得更好笑，
只會變得更識相。

06 · 最難的是閉嘴

昨晚，它想了 103 對

說「講不出來」

77

講得出來

26

四對裡有三對，它選擇閉嘴。這正是我要的。

9/28 19:40–23:44，展示用的 vault，開發模式。

閉嘴有兩層

閘門決定什麼時候講。
判斷決定有沒有東西講。

但做到一半，我發現一件很諷刺的事

我自己的隨筆，
最近都是打在 ChatGPT 裡，
不是 Obsidian。

做筆記精靈的人，不在自己的筆記裡記筆記。

為什麼？

GitHub 這篇，該選哪個 subtype？

Tool？
它是工具

Platform？
它是平台

Organization？
它是微軟的子公司

選哪個都不太對。但 QuickAdd 不讓我不選。

我的知識庫不像專案

我的專案

先有 Project
再長出 Task、Document

結構在前，內容在後

我的知識累積

先有一堆碎片
碎片夠多了，MOC 才自然長出來

內容在前，結構在後

07 • 知識管理到底在管什麼

不是不要結構，是結構晚一點

先留下來
Capture

→

用過幾次
累積夠多

→

再長出結構
MOC、tag、分類

我反對的是：**在證據出現之前，就要求結構先存在。**

`capture first, structure later`

工程師應該都懂的一件事

一個資料庫，不會因為 index 很漂亮、table 很正規化，就變成好的資料庫。

那些事值得做，是因為 query 更可靠、更快。

07 • 知識管理到底在管什麼

不是管知識，也不是管筆記。
是管「讓舊的思考重新變得有用」要付的成本。

Capture
記下時花的腦力

+

Maintenance
平常整理

+

Retrieve
找東西

+

Resume
重新理解當時在想什麼

當然不是越低越好——**不然最省的方法，就是什麼都別記。**

整理多少，是一條軸

完全不整理

3 年後找不到

每篇都整理

懶得記

分類、tag、連 MOC、寫 summary、

填 metadata、提煉 evergreen

不是每篇筆記都值得 Maintenance。

我願意為了一次未來有價值的找回，現在付多少？

回到一開始那句話

一年前	**MOC**	解決「**找得到**」
現在	**精靈**	想解決「**碰得到、接得起來**」

它可以讓我不為了這件事額外整理的情況下，
讓舊的東西自己來碰我。

讓一段現在值得留下的思考，
在未來需要它時，
用最低的總成本**重新變得有用**。